

Algoritmo Test Harnesses

By PhD. Josafá Pontes

Algoritmo Test Harnesses

- Evaluación de modelos de ML
- Train-Test Split
- Cross Validation

¿Por qué Test Harness?

- No sabemos qué algoritmo es mejor
- Necesitamos evaluación consistente
- Comparación justa

Componentes del Test Harness

- Método de muestreo
- Puntero del Algoritmo
- Métrica de evaluación
- Los hiperparámetros del algoritmo

Dataset ejemplo

- Pima Indians Diabetes
- Baseline $\approx$ 65% accuracy

Train-Test Split

- Divide dataset en entrenamiento y prueba
- Simple y rápido

Flujo Train-Test

- Split del dataset
- Entrenar modelo
- Predecir a partir del modelo
- Evaluar el modelo

Implementación

- `evaluate_algorithm()`
- Entrada: dataset, algoritmo, split

Detalles importantes

- Evitar data leakage
- Separar labels en test

Métricas

- Accuracy (clasificación)
- RMSE (regresión)

Ejemplo resultado de Train Test

- Accuracy $\approx$ 65%
- Baseline comparable

Cross Validation

- Divide en k folds
- Evalúa múltiples veces

Flujo Cross Validation

- Iterar k folds
- Train en $k-1$
- Test en 1 fold

Ventajas de la Cross Validation

- Las métricas son más robustas
- Menor varianza a nivel del **resultado promedio**

Costo del Cross Validation

- Más costo computacional (k modelos)
- Más tiempo

Implementación con Cross Validation

- `evaluate_algorithm_cross_validation_split()`
- Retorna lista de scores

Resultados Cross Validation

- Scores múltiples
- Promedio final $\approx 65\%$

Interpretación

- Consistencia entre k folds
- Comparación de modelos

Conclusión

- Test Harness es esencial
- Permite evaluación rigurosa de los modelos de ML